

|                    |                                                                                                          |
|--------------------|----------------------------------------------------------------------------------------------------------|
| <b>Challenge:</b>  | The Portrait                                                                                             |
| <b>Category:</b>   | Steganography                                                                                            |
| <b>Difficulty:</b> | Difficult                                                                                                |
| <b>Tools Used:</b> | Web Browser, Aperi'Solve, CyberChef, <a href="https://cryptii.com/">Cryptii.com</a> , VS Code (optional) |

## Summary:

In this challenge, we are given an image (.png). Using tools like **Aperi'Solve** we analyse the image and look for any hidden zip files. When we find none, we move on to analyse various bit planes and find 2 fake flags in the red and blue channels of bit plane 1. After looking some more we find a string "**5YQ<#6sC**" in bit plane 3. Using **CyberChef** we try standard **Base85** decoding, and fail. Realising there may be an **offset**, we write a python script to extract the bits systematically. Afterwards, we try **Base85** decoding on this binary string but fail again. The bits are actually **flipped** and the bits in each byte are **reversed**. Hence, we write another python script to unflip and reverse the bits. The unflipped binary string is still not standard **Z85**. Finally, we write a python script to create the custom Z85 alphabet and then decode the string to get our flag **MORSMORDRE{p3ns13v3\_dump\_r3tr13v3d}**.

## Problem Statement:

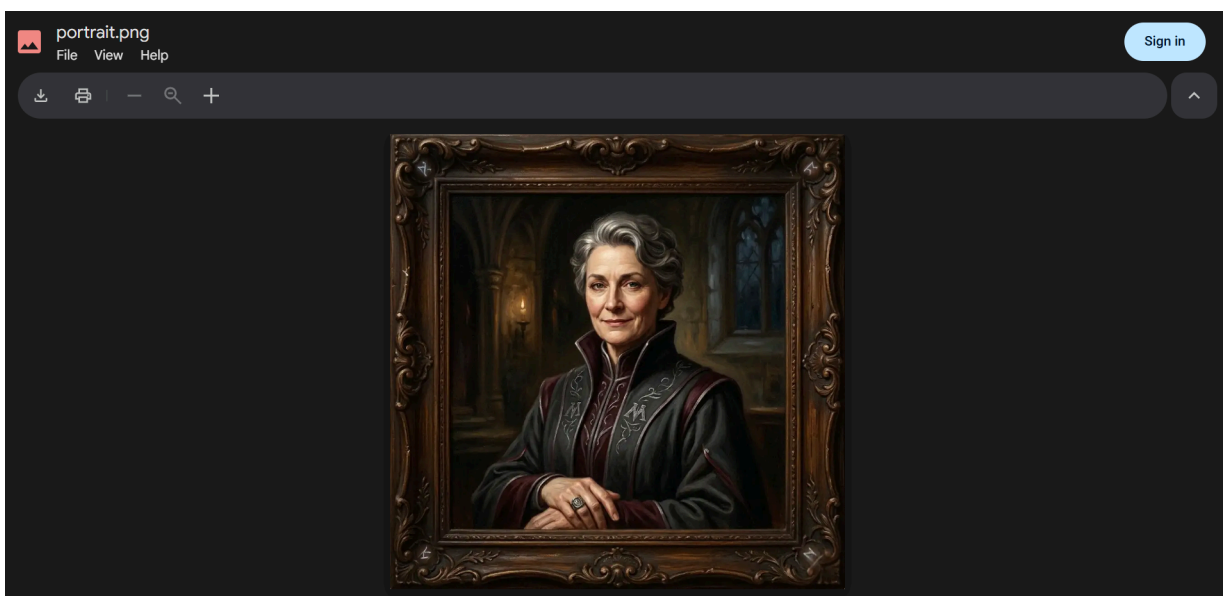
In Hogwarts, every corridor is lined with portraits. Not prints - real paintings, enchanted, inhabited by the spirit of the person depicted. They gossip. They spy. They pass messages between frames. Dumbledore's portrait hangs in the headmaster's office and advises successors long after his death.

This portrait is of a Ministry official. Unnamed. Smiling, the fixed, professional smile of someone who spent their career representing an institution they may not have fully believed in.

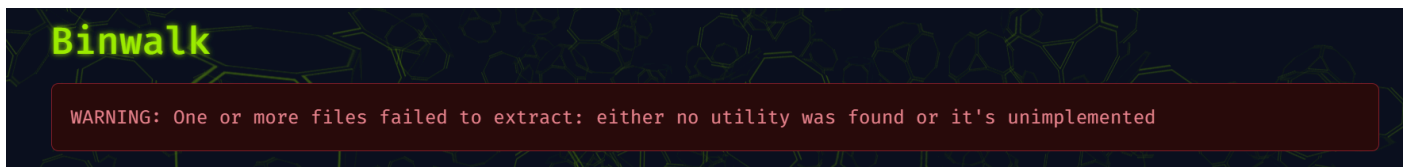
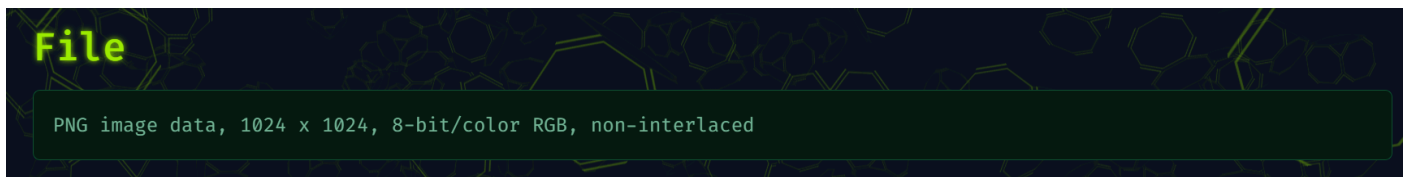
SNAPES used this portrait as a dead drop. Not the magic version - the data version. Something is hidden inside this image in a way that only someone looking for it would find. Find it!!

## Steps to Solve:

1. Open the URL and download the provided image (.png).

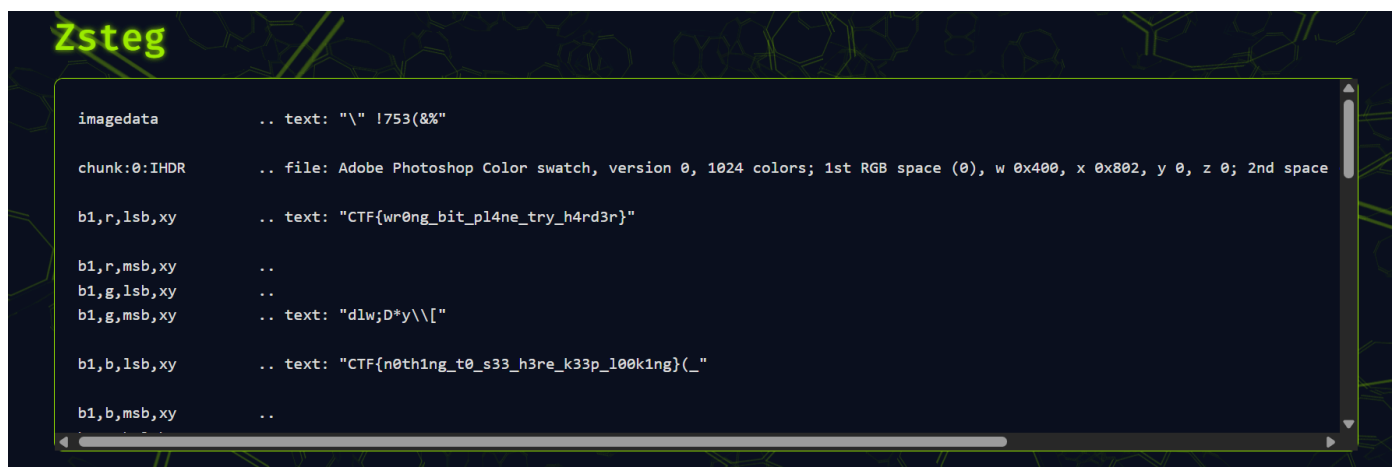


- Open Aperi'Solve, upload this image and click "Analyse Image". Scroll down and analyse:  
**File:** PNG image data, 1024 x 1024, 8-bit/color RGB, non-interlaced  
**Binwalk:** Empty  
**Pngcheck:** IHDR (image header), IDAT (image data) and IEND (image end)  
This means that there is no hidden zip file in the image.

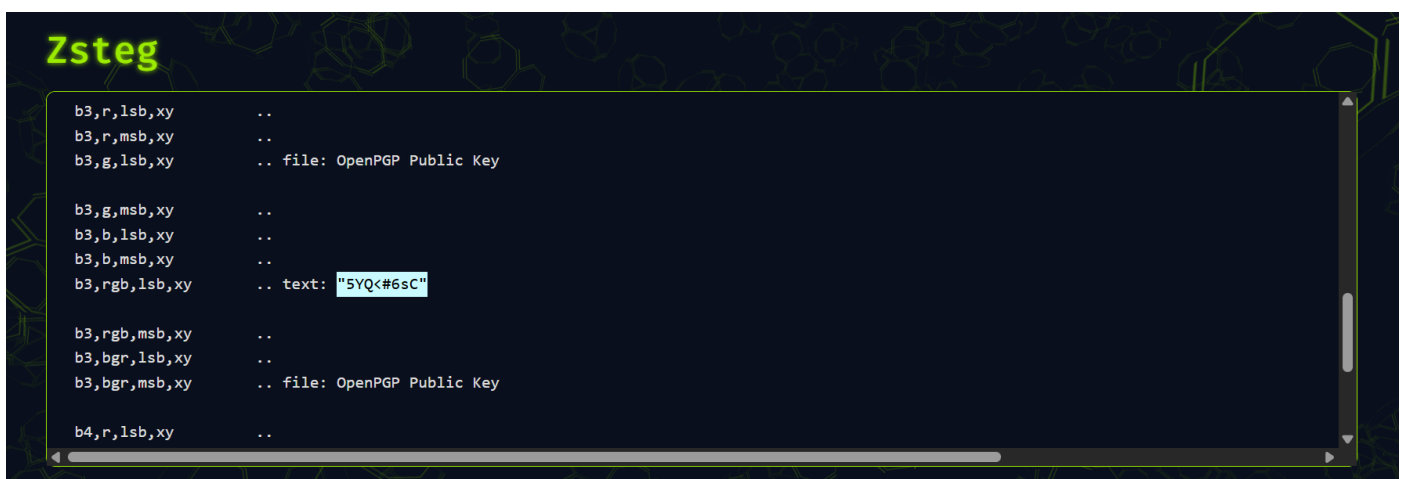


- Scroll down to the "Zsteg" output. Analyse various bit planes. We see:  
Red channel (bit plane 1): **CTF{wr0ng\_bit\_pl4ne\_try\_h4rd3r}**  
Blue channel (bit plane 1): **CTF{n0th1ng\_t0\_s33\_h3re\_k33p\_l00k1ng}{\_**

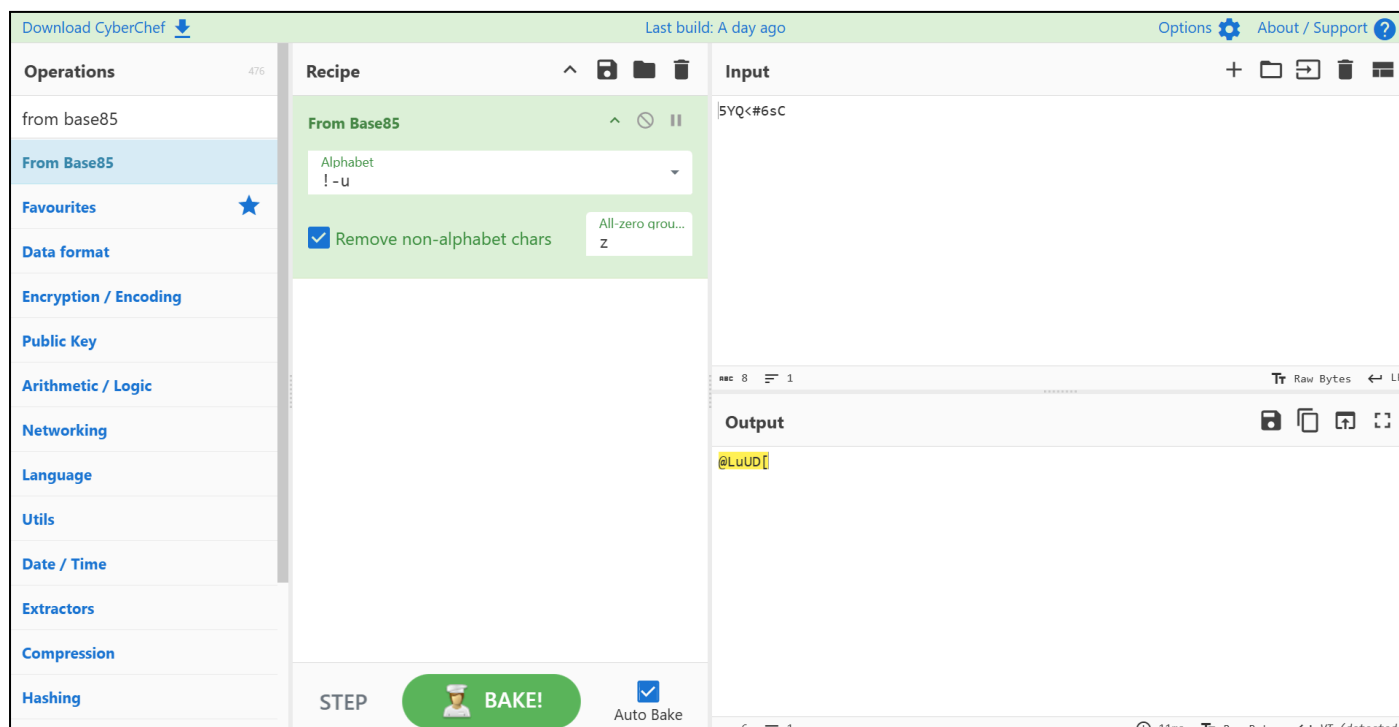
**Hint 1:** Standard tools like **Zsteg** will reveal messages in the lowest bit planes (red 0, red 2, blue 0). Don't trust them. They are decoys. The true message is buried slightly deeper in the red channel, and it doesn't start at the very beginning of the file.



- Scroll down further. In bit plane 3 we find the only clean text "**5YQ<#6sC**" that uses all 3 channels.



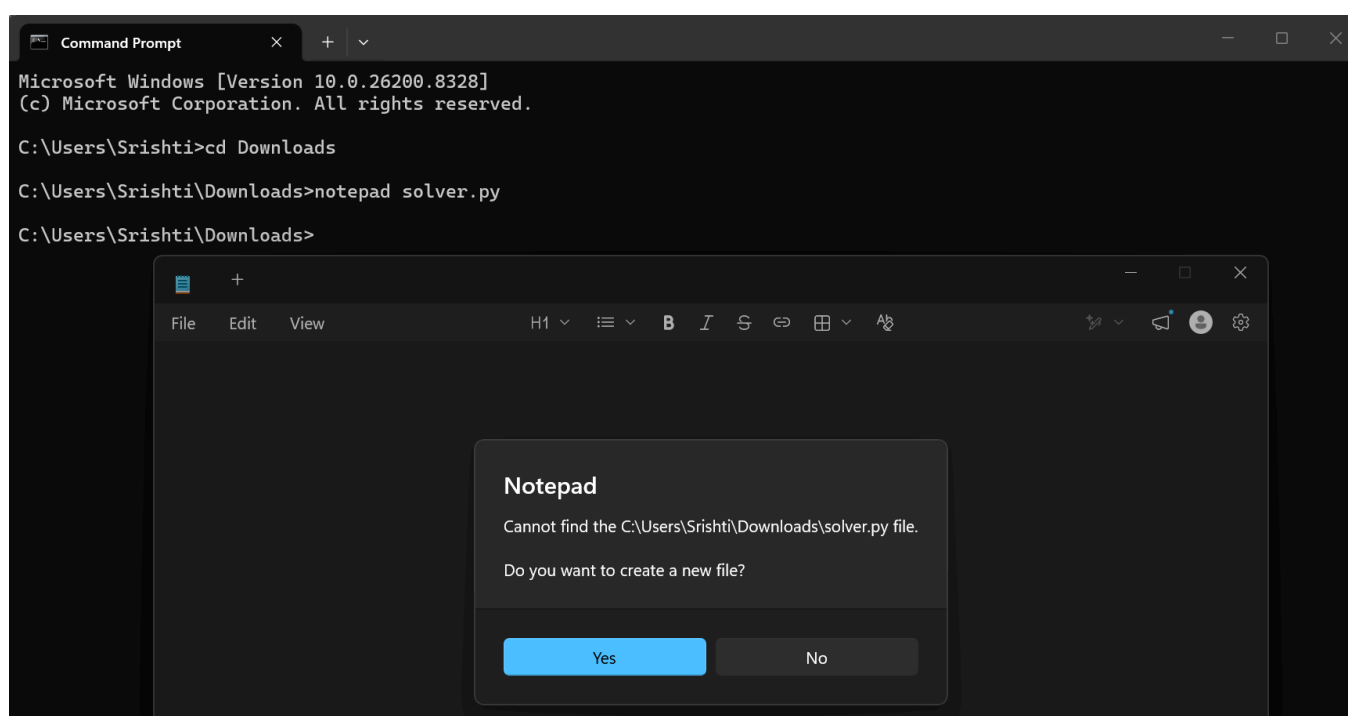
5. Open **CyberChef** online. Paste this string in the input box. Drag and drop the recipe “**From Base85**”. The output is unreadable.



**Hint 2:** The data isn't laid out in a continuous block. If reading from the start yields garbage, consider that the author might have started at an **offset** and **skipped pixels** to avoid detection. You'll need to write a **script** to read the bits systematically rather than relying on automated tools.

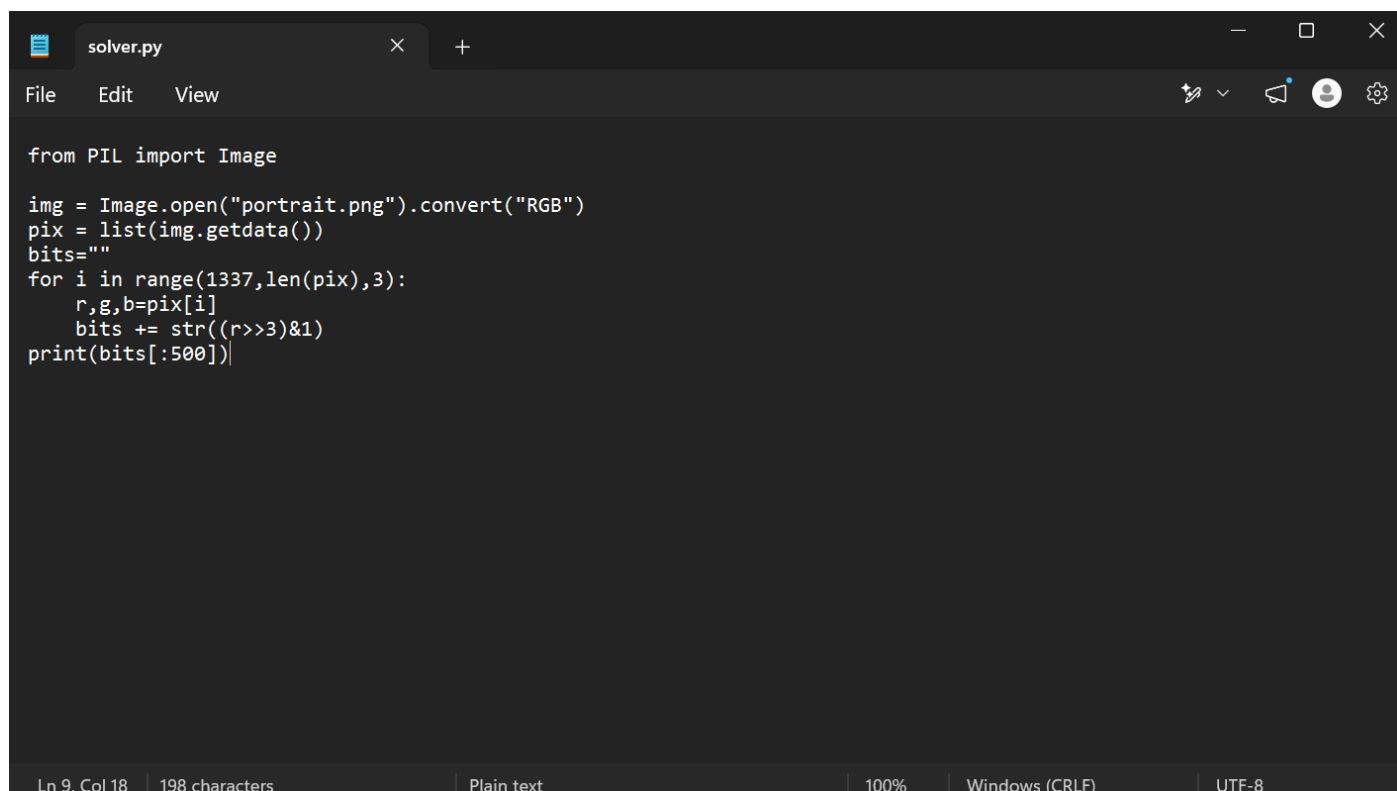
6. Open the terminal (Start > Command Prompt). Create a python file (say [solver.py](#)) in the folder you saved the image in (say Downloads) using the following steps:
- Run the command: **cd Downloads**
  - Then run the command: **notepad solver.py**

A pop-up “Do you want to create a new file?” comes. Click “Yes”.

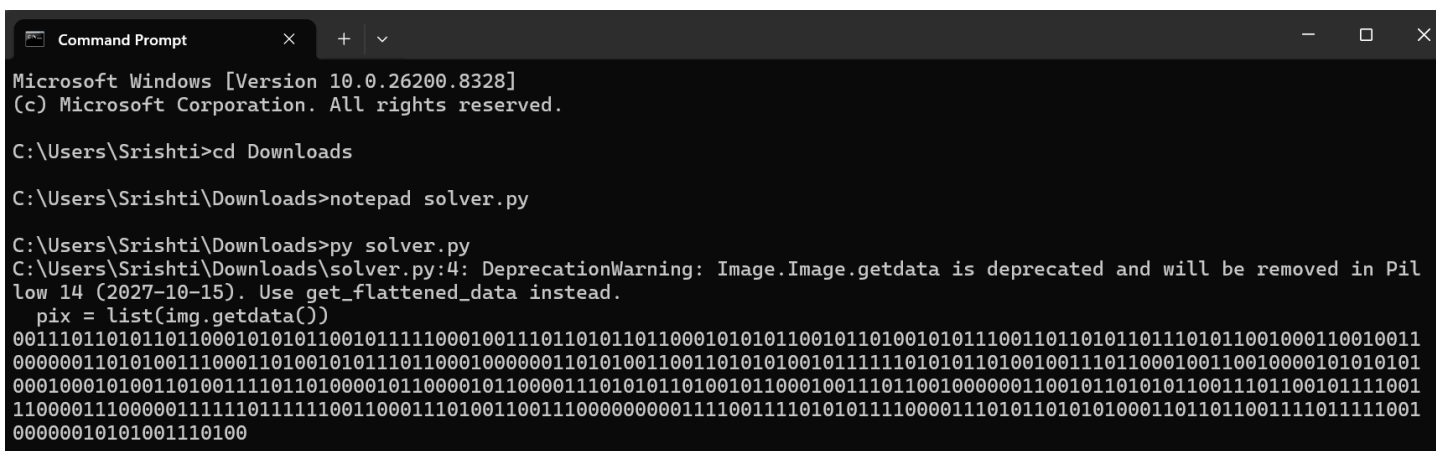


7. In this file write the following python script, save it and close.

```
from PIL import Image
img = Image.open("portrait.png").convert("RGB")
pix = list(img.getdata())
bits=""
for i in range(1337,len(pix),3):
    r,g,b=pix[i]
    bits += str((r>>3)&1)
print(bits[:500])
```



8. In the terminal run the command “**py solver.py**”. This gives a binary string as the output. Copy this output.



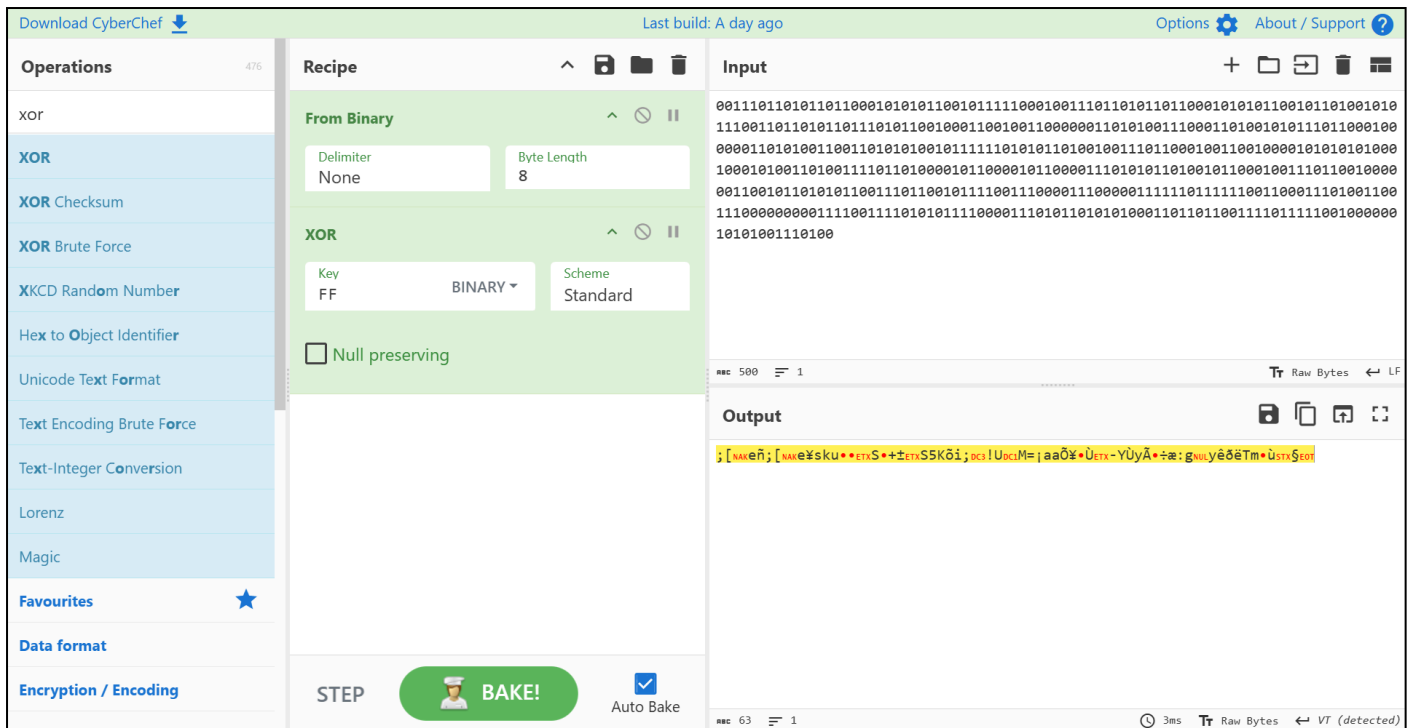
```
Microsoft Windows [Version 10.0.26200.8328]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Srishti>cd Downloads

C:\Users\Srishti\Downloads>notepad solver.py

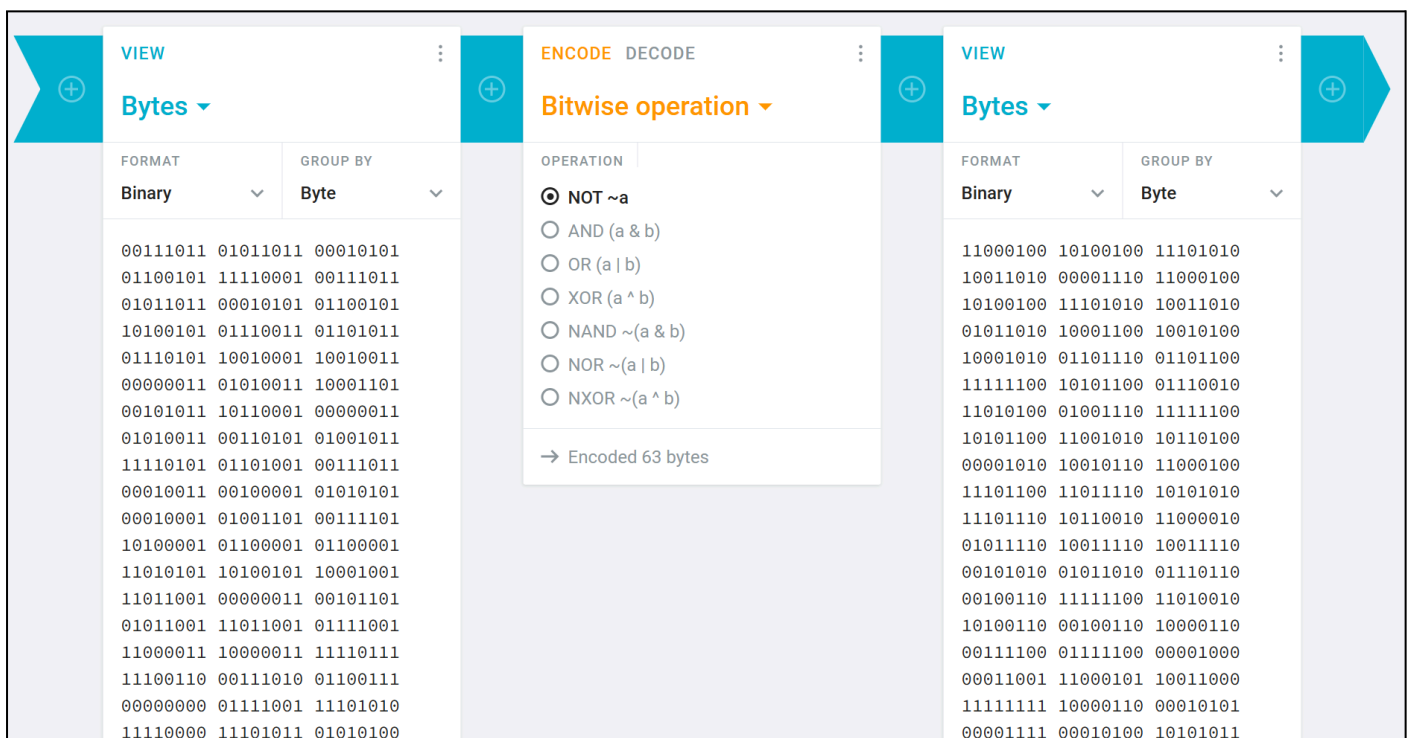
C:\Users\Srishti\Downloads>py solver.py
C:\Users\Srishti\Downloads\solver.py:4: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pil
low 14 (2027-10-15). Use get_flattened_data instead.
  pix = list(img.getdata())
00111011010110110001010101100101111100010011101101011011000101010110010110100101011100110110101101101011001000110010011
000000110101001110001101001010111011000100000011010100110011010101001011111101010110100100111011000100110010000101010101
000100010100110100111101101000010110000101100001110101011010010110001001110110010000001100101101010110011101100101111001
1100001110000011111101111100110001110100110011100000000011110011110101011100001110101101010001101101100111101111001
000000101010011110100
```

9. Open **CyberChef** again and paste the binary string in the input box. Use the following recipes:
- **From Binary**: Set **Byte length**: 8 and **Delimiter**: None
  - **XOR**: Set **Key**: FF and use **Binary**



**Hint 3:** Even when you extract the correct bits, the resulting bytes won't make sense immediately. The author was paranoid: every bit has been flipped (0 becomes 1, 1 becomes 0), and the order of the bits within each individual byte has been reversed. You must undo both transformations to see the encoded string.

- Open [Cryptii.com](https://cryptii.com) and open the dropdown menu next to “**Enigma Machine**”. Under “**Transform**” choose “**Bitwise operation**”. In the input box choose **Bytes**. Select format as **Binary** and paste the binary string. Select the “**NOT ~a**” operation. In the output box, select **Bytes** and format as **Binary**. Copy the output binary string.



- Now, open Command Prompt again and make a new python file (say [solver2.py](#)) by using the command **notepad solver2.py** and then write the following python script in it to reverse the bits in each byte (paste the flipped string in the right place).

```
def reverse_bits_in_byte(b: int) -> int:
    result = 0
    for _ in range(8):
        result = (result << 1) | (b & 1)
        b >>= 1
    return result
```

```
# — 3. Reassemble bytes + undo bit reversal —
unflipped = [int(b) for b in bit_string]
recovered_chars = []
for i in range(0, len(unflipped) - 7, 8):
    byte_bits = unflipped[i:i+8]
    val = 0
    for b in byte_bits:
        val = (val << 1) | b
    val = reverse_bits_in_byte(val)
    recovered_chars.append(val)
```

You can run this in the terminal using “**py** [solver2.py](#)” or you can open it in VS Code by running the command “**code** [solver2.py](#)” in the terminal.

```
def reverse_bits_in_byte(b: int) -> int:
    result = 0
    for _ in range(8):
        result = (result << 1) | (b & 1)
        b >>= 1
    return result

# — 2. Input binary string (space-separated or continuous) —
bits = "11000100 10100100 11101010 10011010 00001110 11000100 10100100 11101010 10011010 01011010 10011010 10011010 10011010 10011010 10011010 10011010"
bit_string = bits.replace(' ', '')

# — 3. Reassemble bytes + undo bit reversal —
unflipped = [int(b) for b in bit_string]
recovered_chars = []
for i in range(0, len(unflipped) - 7, 8):
    byte_bits = unflipped[i:i+8]
    val = 0
    for b in byte_bits:
        val = (val << 1) | b
    val = reverse_bits_in_byte(val)
    recovered_chars.append(val)

# — 4. Trim at null byte —
if 0 in recovered_chars:
    recovered_chars = recovered_chars[:recovered_chars.index(0)]
```

**Hint 4:** The string you finally recover looks like standard Base85 or Z85, but standard decoders will fail. The alphabet isn't standard; it has been shuffled. You will need to recreate the **custom Z85 alphabet** using Python's **random.shuffle()** and a highly predictable, "universal" seed number before decoding the final string.

12. After running the script, write the following python script below the previous script to decode the custom Z85 string.

```
import random

# — 1. Reconstruct the custom Z85 alphabet —
Z85_STANDARD = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ.-:+=^!/*?&<>()[]{}@%$#"
chars = list(Z85_STANDARD)
random.seed(42)
random.shuffle(chars)
Z85_CUSTOM = "".join(chars)

# — 2. Decode using custom Z85 alphabet —

def z85_decode_custom(encoded: str, alphabet: str) -> bytes:
    pad = (5 - len(encoded) % 5) % 5
    encoded = encoded + alphabet[0] * pad
    result = []
    for i in range(0, len(encoded), 5):
        acc = 0
        for c in encoded[i:i+5]:
            acc = acc * 85 + alphabet.index(c)
        result.extend(acc.to_bytes(4, 'big'))
    return bytes(result[:len(result) - pad]) if pad else bytes(result)

# — 3. Build Z85 string, trim invalid chars, pad to multiple of 5 —
clean = ""
for c in ".join(chr(c) for c in recovered_chars):
    if c in Z85_CUSTOM:
        clean += c
recovered_z85 = clean[:len(clean) - (len(clean) % 5)]
print("Z85:", recovered_z85)

# — 4. Decode and print flag —
flag = z85_decode_custom(recovered_z85, Z85_CUSTOM)
print("FLAG:", flag.decode('latin-1').split('}')[0] + '}')
```

Output:

Z85: #%WYp#%WYZ1)Qv6?5N+r?5S-Pi#7{UwMCzyyTZnd?Keda<>a(I

```

21     recovered_chars.append(val)
22
23 # — 4. Trim at null byte —
24 if 0 in recovered_chars:
25     recovered_chars = recovered_chars[:recovered_chars.index(0)]
26
27 import random
28
29 # — 1. Reconstruct the custom Z85 alphabet —
30 Z85_STANDARD = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ.-:+=^!/*?&<>()[]{}@%$#"
31 chars = list(Z85_STANDARD)
32 random.seed(42)
33 random.shuffle(chars)
34 Z85_CUSTOM = ''.join(chars)
35
36 # — 2. Decode using custom Z85 alphabet —
37
38 def z85_decode_custom(encoded: str, alphabet: str) -> bytes:
39     pad = (5 - len(encoded) % 5) % 5
40     encoded = encoded + alphabet[0] * pad
41     result = []
42     for i in range(0, len(encoded), 5):
43         acc = 0
44         for c in encoded[i:i+5]:
45             acc = acc * 85 + alphabet.index(c)
46         result.extend(acc.to_bytes(4, 'big'))
47     return bytes(result[:len(result) - pad]) if pad else bytes(result)
48

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Z85: #%WYp#%WYZ1)Qv6?5N+r?5S-Pi#7{UwMCzyyTZnd?Keda<>a(I  
 FLAG: MORSMORDRE{p3ns13v3\_dump\_r3tr13v3d}

Python Deb...  
 Python Deb...

FLAG: MORSMORDRE{p3ns13v3\_dump\_r3tr13v3d}